

Introduction

- En python, pour le moment on sait créer une variable
- Créer 10 variables, c'est facile.
- Comment on fait pour créer 10000 variables.
- Quelque soit le langage utilisé, nous avons besoin d'un mécanisme permettant de créer, accéder et gérer un nombre quelconque de variables.
- En C/C++, il existe la notion de tableau :
 - Permet de stocker des éléments de même type
 - Taille fixe
 - Très bas niveau, simple mais efficace
- En python, il existe la notion de liste
 - Taille dynamique (on peut ajouter/enlever à volonté).
 - Peut contenir de types différents

- Chaque langage possède ses propres outils
- Ne pas confondre un tableau du C ou C++ avec les listes de python
- En C++, il existe un équivalent aux listes de python : `std::vector`
- Les tableaux de base en C++/Java sont plus "rigides" car leur taille est fixée dès le départ.
 - Par contre ils sont préférables pour une implémentation en temps réel, par exemple.
- Dans la seconde partie de ce semestre, vous allez étudier la notion de tableau en C et C++
 - Les listes de python n'existent pas en C++

Les listes en python

- Les listes sont des objets pouvant contenir d'autres objets.
- Les objets contenus dans une liste peuvent être de types différents : entiers, flottants, chaînes de caractères...

```
1  >>> ma_liste = list() # On crée une liste vide
2  >>> type(ma_liste)
3  <class 'list'>
4  >>> ma_liste
5  []
6  >>>
```

Listes

```
1 >>> ma_liste = [] # On crée une liste vide
2 >>>
```

```
1 >>> ma_liste = [1, 2, 3, 4, 5] # Une liste avec cinq objets
2 >>> print(ma_liste)
3 [1, 2, 3, 4, 5]
4 >>>
```

- Une liste peut contenir des objets de types différents

```
1 >>> ma_liste = [1, 3.5, "une chaine", []]
2 >>>
```

Comment accéder aux éléments d'une liste

```
1  >>> ma_liste = ['c', 'f', 'm']
2  >>> ma_liste[0] # On accède au premier élément de la liste
3  'c'
4  >>> ma_liste[2] # Troisième élément
5  'm'
6  >>> ma_liste[1] = 'Z' # On remplace 'f' par 'Z'
7  >>> ma_liste
8  ['c', 'Z', 'm']
9  >>>
```

- Les listes sont des objets modifiables, elles sont dites ***mutable***
- Ajouter un élément dans une liste :

```
1  >>> ma_liste = [1, 2, 3]
2  >>> ma_liste.append(56) # On ajoute 56 à la fin de la liste
3  >>> ma_liste
4  [1, 2, 3, 56]
5  >>>
```

Les listes sont des objets modifiables

- La plupart des fonctions appliquées à une liste ne renvoie rien, mais modifie la liste elle-même.

```
10 ... liste1 = [1, 5.5, 18]
11 >>> liste2 = liste1.append(-15) # On ajoute -15 à liste1
12 >>> liste1                        # On affiche liste1
13 [1, 5.5, 18, -15]
14 >>> # Cette fois, l'appel de la méthode a modifié l'objet d'
    origine (liste1)
15 ... # Voyons ce que contient liste2
16 ... liste2
17 >>> # Rien ? Vérifions avec print
18 ... print(liste2)
19 None
20 >>>
```

- Ajouter un élément à un rang donné dans une liste.

```
1 >>> ma_liste = ['a', 'b', 'd', 'e']
2 >>> ma_liste.insert(2, 'c') # On insère 'c' à l'indice 2
3 >>> print(ma_liste)
4 ['a', 'b', 'c', 'd', 'e']
```

- Concaténations de listes :

```
1 >>> ma_liste1 = [3, 4, 5]
2 >>> ma_liste2 = [8, 9, 10]
3 >>> ma_liste1.extend(ma_liste2) # On insère ma_liste2 à la fin
  de ma_liste1
4 >>> print(ma_liste1)
5 [3, 4, 5, 8, 9, 10]
6 >>> ma_liste1 = [3, 4, 5]
7 >>> ma_liste1 + ma_liste2
8 [3, 4, 5, 8, 9, 10]
9 >>> ma_liste1 += ma_liste2 # Identique à extend
10 >>> print(ma_liste1)
11 [3, 4, 5, 8, 9, 10]
12 >>>
```

- Suppression d'un élément d'une liste :

```
1 >>> ma_liste = [-5, -2, 1, 4, 7, 10]
2 >>> del ma_liste[0] # On supprime le premier élément de la
   liste
3 >>> ma_liste
4 [-2, 1, 4, 7, 10]
5 >>> del ma_liste[2] # On supprime le troisième élément de la
   liste
6 >>> ma_liste
7 [-2, 1, 7, 10]
8 >>>
```

- La méthode remove :

```
1 >>> ma_liste = [31, 32, 33, 34, 35]
2 >>> ma_liste.remove(32)
3 >>> ma_liste
4 [31, 33, 34, 35]
5 >>>
```


Le parcours de listes

```
1  >>> ma_liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
2  >>> i = 0 # Notre indice pour la boucle while
3  >>> while i < len(ma_liste):
4  ...     print(ma_liste[i])
5  ...     i += 1 # On incrémente i, ne pas oublier !
6  ...
7  a
8  b
9  c
10 d
11 e
12 f
13 g
14 h
15 >>> # Cette méthode est cependant préférable
16 ... for elt in ma_liste: # elt va prendre les valeurs
    successives des éléments de ma_liste
17 ...     print(elt)
18 ...
19 a
20 b
21 c
22 d
23 e
24 f
25 g
26 h
27 >>>
```

La fonction enumerate

```
1  >>> ma_liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
2  >>> for i, elt in enumerate(ma_liste):
3  ...     print("À l'indice {} se trouve {}".format(i, elt))
4  ...
5  À l'indice 0 se trouve a.
6  À l'indice 1 se trouve b.
7  À l'indice 2 se trouve c.
8  À l'indice 3 se trouve d.
9  À l'indice 4 se trouve e.
10 À l'indice 5 se trouve f.
11 À l'indice 6 se trouve g.
12 À l'indice 7 se trouve h.
13 >>>
```

- Ecrire une fonction qui cherche un élément x dans une liste.

```
def inn(l,x):  
    for e in l:  
        if(e==x): return True  
    return False
```

```
l=[1,2,3,4]
```

```
print inn(l,3)  
print inn(l,10)
```

version avec while

```
def inn(l,x) ;  
    n=len(l)  
    i = 0  
    while i<n :  
        if(l[i]==x) :  
            return True  
    return(False)
```

- Ecrire une fonction qui retourne l'indice du plus grand élément dans une liste :

```
def pg(l):  
    ind=0  
    pge=l[0]  
    for i in xrange(0,len(l)):  
        if(l[i]>pge):  
            pge=l[i]  
            ind=i  
  
    return ind
```

```
L=[5,7,9,3,2]
```

```
print pg(L)
```

- Ecrire une fonction qui calcule la moyenne des valeurs qui se trouvent dans une liste :

```
def moyenne(l):  
    som=0.0  
    for i in xrange(0,len(l)):  
        som+=l[i]  
    return som/len(l)
```

```
L=[5,7,9,3,2]
```

```
print moyenne(L)|
```


- Ecrire une fonction qui retourne l'indice du plus grand élément parmi les i premiers éléments d'une liste.
- Utiliser cette fonction pour trier une liste donnée

```
def indice(l,i):  
    maxi=l[0]  
    for j in xrange(i):  
        if(l[j]>l[maxi]): maxi=j  
  
    return maxi
```

```
def trier(l):  
    n=len(l)  
    for j in xrange(n-1):  
        imax=indice(l,n-j)  
        l[imax],l[n-j-1]=l[n-j-1],l[imax]
```

```
l=[1,2,7,3,-1]  
trier(l)  
print(l)
```

- Ecrire une fonction qui renvoie la médiane d'une série de valeurs contenues dans une liste :

```
def mediane(l):  
    trier(l)  
    return l[len(l)/2]
```

- Ecrire une fonction renvoie la liste des indices ou se trouve un élément donné dans une liste donnée

```
def recherche_indices(l,x):  
    l_indices=[]  
    for i in xrange(len(l)):  
        if (l[i]==x):  
            l_indices.append(i)  
  
    return l_indices
```

```
l=[1,5,8,9,1,1]
```

```
l1=recherche_indices(l,1)  
print l1
```

- Ecrire une fonction qui insère un élément dans une liste triée.

```

def insere_trie(l,x):
    n=len(l)
    if (x <= l[0]):
        l.insert(0,x)
        return
    if (x >= l[n-1]):
        l.insert(n,x)
        return

    d=0
    f=n-1
    while (f-d) > 1:
        m=(f+d)/2
        if(x<l[m]):
            f=m

        else:
            d=m

    l.insert(d+1,x)

l=[1,2,4,5]
x=3
insere_trie(l,2)
print l

```